
README + AI_USAGE Report

Candidate: Pruthvi R | CipherSchools 2-Day Engineering Challenge

AI_USAGE: Key Architectural Decisions & AI Partnership

Decision 1: Hybrid Evaluation Pipeline vs. Pure LLM Prompting

- AI Suggestion: Pass full solution directly to an LLM prompt for single-shot JSON evaluation.
- Engineering Decision: Rejected pure LLM; implemented a Hybrid Strategy Pipeline instead.
- Rationale: Mitigates 3s latency, eliminates API quota bloat, and solves non-deterministic scoring variations by running deterministic AST/keyword checks alongside qualitative AI reasoning.

Decision 2: Interactive Code Workspace vs. Visual UML Drag-and-Drop Canvas

- AI Suggestion: Build a React Flow visual node-based UML diagramming canvas.
- Engineering Decision: Rejected visual diagrammer; adopted Code/Pseudocode Editor.
- Rationale: Real-world engineering LLD interviews require implementing actual classes, interfaces, and access modifiers.

Decision 3: Timeout Protection & Resilient Fallback

- AI Suggestion: Add complex background worker queues with polling/webhooks.
- Engineering Decision: Accepted client-side Promise.race timeout (4,000ms) with graceful fallback.
- Rationale: Complies with assignment boundary ("skip distributed systems overhead") while guaranteeing 0% failure rate for learners.

Decision 4: Attempt History & Score Progression

- AI Suggestion: Overwrite state with only the latest attempt.
- Engineering Decision: Rejected; engineered full AttemptRepository tracking iterations and score deltas (+15 pts).

Project Quick Start & Execution Guide

Prerequisites: Node.js (v18+) and npm.

- Step 1: Install dependencies using `npm install`
- Step 2: Start local development server with `npm run dev` (running at `http://localhost:5173`)
- Step 3: Run automated test suite using `npm test`
- Step 4: Build production bundle using `npm run build`

Automated Testing & Reliability

The platform includes a Vitest suite covering:

- Deterministic parser handling of empty/trivial submissions (returns Incomplete verdict).
- Pattern recognition: Strategy, Singleton, and State pattern detection in reference solutions.
- SOLID metric synthesis across Single Responsibility, Open/Closed, and Dependency Inversion.
- Hybrid pipeline coordination with timeout boundary testing.